

Exercise: Matrix Multiplication — Naive vs. NumPy, and Where the FLOPs Go

Language: Python 3 + NumPy
Topics: FLOP counting, empirical benchmarking

Learning Objectives

By the end of this exercise, you should be able to:

1. Derive the FLOP count for matrix multiplication from its algorithmic definition.
 2. Compute **GFLOP/s** (achieved performance) from timing data.
 3. Explain — with evidence, not just intuition — why a naive triple-loop implementation can be far slower than NumPy's `@` operator despite doing the *same number* of floating-point operations.
-

Part 0: Setup

Go to Github and clone the repo. Then, browse to **Matmul_and_GFLOP**. You'll need NumPy and `time` (both standard). No GPU required.

```
import numpy as np
import time
```

Part 1: Derive the FLOP Formula (Pen and Paper)

Consider multiplying two matrices: A of shape (M, K) and B of shape (K, N) , producing $C = AB$ of shape (M, N) .

- 1.1.** Write the formula for a single output element C_{ij} in terms of A and B .

- 1.2. How many multiplications and how many additions does computing **one** element C_{ij} require? (Be careful with the additions — how many terms are you summing?)
- 1.3. How many total elements does C have?
- 1.4. Combine 1.2 and 1.3 to derive a formula for the **total number of FLOPs** to compute $C = AB$, in terms of M , K , N . Express it counting *both* multiplications and additions as separate FLOPs (this is the standard convention).
- 1.5. For the special case of two square $n \times n$ matrices, simplify your formula in terms of n alone. What is the asymptotic complexity in Big-O terms?
-

Part 2: Implement Naive Matrix Multiplication

- 2.1. We will be working on `Matmul_and_GFLOP/main.py`. Your task is to implement matrix multiplication **using three nested Python loops** — no NumPy operations inside the loop body (you may use NumPy only to *allocate* the zero-initialized output array and to hold inputs as arrays). **Put your answer in the python file.**

```
def matmul_naive(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """
    Multiply A (M x K) and B (K x N) using explicit triple-nested
        loops.
    Returns C (M x N).
    """
    M, K = A.shape
    K2, N = B.shape
    assert K == K2, "Inner dimensions must match"

    C = np.zeros((M, N))

    # TODO: implement with three nested for-loops
    # for i in range(M):
    #     for j in range(N):
    #         for k in range(K):
```

```
# ...

return C
```

2.2. Verify correctness against NumPy on a small random example:

```
A = np.random.rand(10, 12)
B = np.random.rand(12, 8)

C_naive = matmul_naive(A, B)
C_numpy = A @ B

assert np.allclose(C_naive, C_numpy), "Mismatch!"
print("Naive implementation verified correct.")
```

Part 3: Benchmark and Compute Achieved GFLOP/s

3.1. Write a timing harness that runs each implementation on square matrices of size $n \times n$ for $n \in \{64, 128, 256, 512\}$, and records wall-clock time. Use `time.perf_counter()`. For the naive version, a single run per size is fine (it's slow); for NumPy, average over several runs since each run is fast.

```
def time_it(fn, A, B, repeats=1):
    start = time.perf_counter()
    for _ in range(repeats):
        result = fn(A, B)
    elapsed = (time.perf_counter() - start) / repeats
    return elapsed, result

sizes = [64, 128, 256, 512]
results = [] # list of dicts: {n, time_naive, time_numpy}

for n in sizes:
    A = np.random.rand(n, n)
    B = np.random.rand(n, n)

    t_naive, _ = time_it(matmul_naive, A, B, repeats=1)
    t_numpy, _ = time_it(lambda X, Y: X @ Y, A, B, repeats=5)

    results.append({"n": n, "time_naive": t_naive, "time_numpy":
                    t_numpy})
    print(f"n={n:4d}  naive={t_naive:.4f}s  numpy={t_numpy:.6f}s
          speedup={t_naive/t_numpy:8.1f}x")
```

Note: $n = 512$ naive may take a minute or more in pure Python. If it's impractical on your machine, cap at $n = 256$ and note this in your writeup.

3.2. For **each** matrix size, using your formula from Part 1.5, compute:

- The total FLOP count for that size.
- **Achieved GFLOP/s** for each implementation: $\text{GFLOP/s} = \frac{\text{FLOPs}}{\text{time (seconds)} \times 10^9}$

Present this as a table (you can build it in code or by hand):

n	FLOPs	t_{naive} (s)	GFLOP/s (naive)	t_{numpy} (s)	GFLOP/s (numpy)	Speedup
64						
128						
256						
512						

Part 4: Analysis Questions

- 4.1.** Your naive and NumPy implementations compute the **same FLOP count** at each size (verify this is true by construction — both implement the textbook algorithm). Yet GFLOP/s differs by orders of magnitude. What does this tell you about the relationship between FLOP count and actual runtime? Is FLOP count alone a reliable predictor of execution time? Why or why not?
- 4.2.** Look up (or estimate from your CPU's specs) a rough **peak GFLOP/s** for a single core of your machine. How close does NumPy's achieved GFLOP/s get to that peak? What fraction of peak is the naive implementation achieving?
- 4.3.** NumPy's @ calls into a BLAS library (e.g., OpenBLAS, MKL) written in optimized C/Fortran with techniques like **loop blocking/tiling**, **vectorization (SIMD)**, and **multithreading**. In 3–5 sentences, explain in your own words why these techniques close the gap between achieved and peak performance, and why naive triple-loop Python code fails to benefit from them.